DAY 3 • MACHINE LEARNING & AI ENGINEERING

Reinforcement Learning with Large Language Models

From Language Fundamentals & vLLM Serving to PPO, GRPO, GSPO, and SAPO

🎓 College Engineering Guide

33 Interactive Slides • Hands-on AI Theory

OVERVIEW

Day 3 Roadmap & Learning Modules

Mastering the stack from token generation to modern RL post-training algorithms

1. LLM Preliminaries

Tokens, embeddings, transformers, and next-token generation.

2. RL & Sampling

Agents, environments, temperature, top-k, and nucleus sampling.

3. RL Mathematics

Expected rewards, log probabilities, and policy gradients simplified.

4. Hardware Fundamentals

GPU memory limits, VRAM bottlenecks, and parallel processing.

5. vLLM Engine

PagedAttention, continuous batching, and KV cache optimization.

6. PPO, GRPO, GSPO & SAPO

Modern policy optimization algorithms compared step-by-step.

MODULE 1

Language Model Preliminaries

How neural networks process human language: from raw characters to vectors, attention matrices, and probabilistic text output.

MODULE 1 • BASICS

What is a Large Language Model (LLM)?

An artificial neural network trained to predict the most likely next word

Statistical Predictor

An LLM does not "think" like a human. It calculates probabilities across billions of vocabulary words to complete a sequence.

Massive Scale

Trained on trillions of text tokens from books, code, and web pages using billions of learnable parameters.

Emerging Capability

Scale enables zero-shot reasoning, code generation, translation, and structured problem-solving.

MODULE 1 • DATA REPRESENTATION

Tokens and Tokenization

Converting raw human text into numeric chunks for neural processing

Computers cannot process raw characters directly. Tokenizers break text into standard sub-word units.

```
"Reinforcement Learning" → ["Re", "inforce", "ment", " Learn", "ing"]
Token IDs: [4821, 892, 412, 2391, 318]
```

Rule of thumb: 1 token ≈ 0.75 words in English (4 characters).

Tokenizer Types

- **Byte-Pair Encoding (BPE):** Merges frequent character pairs.
- **WordPiece:** Optimized for vocabulary efficiency.
- **SentencePiece:** Handles multi-lingual unsegmented text.

MODULE 1 • VECTORS

Embeddings: Mapping Words to Geometry

Tokens become dense numerical vectors capturing semantic meanings

Dense Representation

Each token ID maps to a high-dimensional vector (e.g., 4,096 numbers) carrying semantic properties.

Semantic Proximity

Words with similar meanings (e.g., "king" and "queen") sit close together in vector space.

Vector Arithmetic

Mathematical relationships hold true: `Vector("King") - Vector("Man") + Vector("Woman") ≈ Vector("Queen")`.

MODULE 1 • CORE MECHANISM

Next-Token Prediction Engine

Autoregressive generation generates text one single token at a time



The output token "blue" is appended back to the input prompt, and the process repeats to generate token #2.

MODULE 1 • LIFECYCLE

Training vs. Inference

Distinguishing weight update learning from static forward generation

Dimension	Training (Pre-training / RL)	Inference (Generation)
Objective	Learn parameters (weights) via backpropagation	Generate text using fixed model weights
Compute Needs	Thousands of GPUs for weeks or months	Single GPU or CPU cluster for seconds
Memory	Stores weights, gradients, optimizer states	Stores weights and KV Cache only
Data Flow	Forward pass + Loss calculation + Backward pass	Forward pass autoregressive loop only

MODULE 1 • ARCHITECTURE

Transformers and Self-Attention (Basic Level)

How LLMs dynamically connect words regardless of spatial distance

Self-Attention

Calculates relevance scores between every pair of words in a sentence (e.g., connecting "it" to "animal" rather than "street").

Multi-Head Design

Multiple attention heads simultaneously track grammar, tone, factual relationships, and context positioning.

Parallel processing

Unlike old RNNs, transformers process entire text prompts at once during pre-training.

MODULE 2

Reinforcement Learning & Sampling

Steering text output through reward feedback and mastering token probability distributions.

MODULE 2 • RL BASICS

What is Reinforcement Learning (RL)?

Learning optimal behavior through trial, error, and scalar rewards

Goal-Driven Learning

Instead of providing ground-truth answers, we rate actions with scalar reward scores (positive or negative).

Feedback Loop

The decision maker interacts with the world, collects rewards, and updates its strategy over time.

LLM Application

LLMs use RL to align generated text with human preferences for helpfulness, accuracy, and safety.

MODULE 2 • CORE RL TERMINOLOGY

Agent, Environment, State, Action, Reward, Policy

Mapping classic reinforcement learning definitions directly to LLMs

RL Concept	General RL Meaning	LLM Specific Equivalent
Agent	The decision maker	The Large Language Model (e.g., Llama, Qwen)
Environment	The world interacting with agent	The context window + user prompt context
State (s)	Current situation	Prompt + tokens generated so far
Action (a)	Decision made by agent	The next output token selected
Reward (r)	Feedback score	Human or automated evaluator score (+1 / -1)
Policy (π)	Strategy/ruleset	LLM parameter weights mapping tokens to probabilities

MODULE 2 • ANALOGY

Game Analogy: Pac-Man vs. Language Models

Understanding sequential decision-making through gameplay

Pac-Man Game

- **State:** Board positions of dots and ghosts.
- **Action:** Move Up, Down, Left, or Right.
- **Reward:** +10 per dot, +200 per ghost, -500 for dying.
- **Goal:** Maximize total game score.

Generating an Essay

- **State:** User prompt + essay sentences written so far.
- **Action:** Pick token from 100,000 possibilities.
- **Reward:** High score for clear logic & factual truth.
- **Goal:** Maximize helpfulness rating.

MODULE 2 • TOKEN SELECTION

What is Sampling in Language Models?

Selecting output tokens from vocabulary probability distributions

Logits to Probabilities

The LLM outputs raw numbers (logits) for all vocabulary words. Softmax turns them into probabilities summing to 1.0 (100%).

Greedy Search

Always selects the single highest-probability token. Leads to repetitive, robotic, and uncreative sentences.

Stochastic Sampling

Samples tokens randomly according to their probability weights, introducing diversity, creativity, and natural flow.

MODULE 2 • HYPERPARAMETERS

Controlling Output: Temperature, Top-K, Top-P

Fine-tuning output randomness, creativity, and structural focus

Temperature (T)

Divides logits before softmax.

- **Low (0.2):** Focused & factual.
- **High (0.9):** Creative & diverse.

Top-K Sampling

Limits selection strictly to the top K most probable candidate tokens (e.g., $K=50$), discarding tail noise.

Top-P (Nucleus)

Selects the smallest set of tokens whose cumulative probability exceeds threshold P (e.g., $P=0.90$).

MODULE 3

Reinforcement Learning Mathematics

Understanding policy gradients, expected rewards, and optimization formulas without numerical overwhelming.

MODULE 3 • MATH FOUNDATIONS

Rewards, Policies & Expected Return

Formulating the primary objective of RL optimization in simple terms

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

Symbol Breakdown

- **θ (Theta):** Learnable weights of the LLM.
- **τ (Tau):** Generated sequence of tokens.
- **π_θ:** Model policy giving token probabilities.
- **$R(\tau)$:** Scalar reward score given to completion.
- **$\mathbb{E}$ (Expectation):** Average reward across all outputs.

In Plain Words

"Adjust the model parameters (θ) so that, on average, sentences generated by the model achieve the highest possible score from the reward function."

MODULE 3 • OPTIMIZATION

The Policy Gradient Trick (REINFORCE)

How we increase the probability of good outputs and suppress bad ones

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E} \left[\nabla_\theta \log \pi_\theta(y|x) \cdot R(y) \right]$$

👍 Positive Reward ($R > 0$)

Increases log probabilities of the generated tokens, making the model more likely to repeat that style in future responses.

👎 Negative Reward ($R < 0$)

Decreases probabilities of generated tokens, teaching the model to avoid generating similar mistakes.

MODULE 3 • ALIGNMENT

Why Language Models Need Reward Signals

Pre-training learns language; RLHF/RLAIF learns alignment and reasoning

⚠ Pre-training Limits

A purely pre-trained model mimics internet text, including hallucinations, toxicity, and unhelpful conversational patterns.

⚖ Non-Differentiable Feedback

Human criteria (helpfulness, tone, correctness) cannot be calculated via simple supervised cross-entropy loss.

☑ Complex Reasoning

RL rewards correct final answers in math and coding, teaching multi-step chain-of-thought reasoning paths.

MODULE 4

Reinforcement Learning on Hardware

GPUs, VRAM constraints, parallel computation, and memory bottlenecks in large model serving.

MODULE 4 • COMPUTE SYSTEMS

CPUs vs. GPUs in Deep Learning

Why matrix multiplication requires specialized massively parallel processors

Central Processing Unit (CPU)

- Optimized for complex sequential control logic.
- Few high-performance powerful cores (e.g., 16–64).
- High latency for massive matrix operations.

Graphics Processing Unit (GPU)

- Optimized for massive parallel tensor math.
- Thousands of smaller compute cores (e.g., 16,000+ Tensor Cores).
- High bandwidth memory (HBM) for fast tensor access.

MODULE 4 • VRAM ALLOCATION

Why GPU Memory (VRAM) Matters Most

Understanding parameter footprints, precision, and active activations

Model Weights

A 70B parameter model in 16-bit precision requires **140 GB** of VRAM just to fit the weights into memory.

KV Cache Space

Stores key and value attention tensors for previous tokens to avoid recomputing past text during generation.

Compute vs. Memory Bound

LLM generation is memory-bandwidth bound: memory transfer speed limits generation speed, not raw FLOPS.

MODULE 4 • SCALING

Distributed Computing Techniques

Splitting huge language models across multiple interconnected GPUs

Data Parallelism

Copies the full model to every GPU and splits training data batches across GPUs.

Tensor Parallelism

Splits individual linear projection matrix operations inside attention layers across GPUs.

Pipeline Parallelism

Distributes transformer layers sequentially across multiple GPUs (e.g., GPU 1 holds layers 1-16, GPU 2 holds 17-32).

MODULE 5

The vLLM Inference Engine

High-throughput LLM serving via PagedAttention, continuous batching, and KV cache memory management.

MODULE 5 • SERVING TECH

What is vLLM & PagedAttention?

Eliminating memory fragmentation in LLM inference servers

⚠ Traditional KV Cache Problem

Pre-allocates contiguous memory for maximum sequence lengths. Up to **60%–80% of VRAM is wasted** due to fragmentation and unused space.

PagedAttention Solution

Inspired by OS virtual memory. Stores KV cache in non-contiguous physical memory blocks. Reduces memory waste down to **under 4%**.

Continuous Batching & Serving Architecture

Dynamic request iteration scheduling maximizes system throughput

Continuous Batching

Instead of waiting for an entire batch to finish, new requests join the iteration batch immediately as completed ones exit.

Key Serving Metrics

- **Throughput:** Tokens per second generated across all users.
- **Latency:** Time To First Token (TTFT) and Inter-Token Latency (ITL).

MODULE 6 & 7

PPO, GRPO, GSPO, and SAPO

Comparing modern policy optimization algorithms for LLM post-training and reasoning alignment.

Proximal Policy Optimization (PPO) vs. GRPO

Moving from complex critic networks to efficient group relative metrics

Classic PPO

Requires **4 separate models** in VRAM during training: Actor, Critic, Reference Model, and Reward Model. High memory footprint.

GRPO (Group Relative)

Eliminates the Critic network! Samples a group of outputs per prompt and calculates advantages relative to the group mean score.

GSPO and SAPO: Addressing Clipping Bottlenecks

Sequence-level ratios and smooth temperature-controlled gating mechanisms

GSPO (Group Sequence)

Replaces token-level ratios with **sequence-level importance ratios**. Stabilizes mathematical reasoning RL training.

SAPO (Soft Adaptive)

Replaces rigid hard clipping with a **smooth sigmoid gating function**, allowing controlled off-policy learning.

MODULE 7 • COMPREHENSIVE TABLE

PPO vs. GRPO vs. GSPO vs. SAPO Comparison

High-level architectural trade-offs across modern LLM RL post-training algorithms

Algorithm	Critic Model Required?	Optimization Granularity	Clipping / Gate Type
PPO	Yes (Large Memory)	Token-level	Hard Clipping $[1-\epsilon, 1+\epsilon]$
GRPO	No (Group Relative)	Token-level	Hard Clipping $[1-\epsilon, 1+\epsilon]$
GSPO	No (Group Relative)	Sequence-level	Sequence-level Hard Clip
SAPO	No (Group Relative)	Token / Sequence Adaptive	Smooth Sigmoid Soft Gate

MODULE 8 • SYNTHESIS

Complete LLM Training & Inference Lifecycle

Connecting tokenization, hardware, vLLM serving, and RL post-training

1. Pre-training → 2. SFT Alignment → 3. RL (GRPO/SAPO) → 4. vLLM Serving

Raw internet data produces a base model; SFT teaches task structure; RL post-training (PPO/GRPO/GSPO/SAPO) refines complex reasoning; vLLM serves requests efficiently in production.

MODULE 8 • ASSESSMENT

Concept Check & MCQs

Test your understanding of Day 3 engineering concepts

Q1. What primary memory bottleneck does vLLM PagedAttention solve?

A) GPU clock speeds
B) KV Cache memory fragmentation
C) Tokenizer speed

Q2. Why does GRPO require significantly less VRAM than classic PPO?

A) Eliminates the Critic model
B) Uses smaller token vocabularies
C) Runs purely on CPUs

DAY 3 COMPLETE

Key Takeaways & Next Steps

You now understand the mathematical and architectural foundations of RL post-training for modern LLMs!

Tokens & Sampling

Probabilistic next-token generation controlled via temperature.

Hardware & vLLM

PagedAttention optimizes KV cache for high-throughput serving.

Modern RL Optimization

GRPO, GSPO, and SAPO provide efficient policy updates for LLMs.